The destructor of `thread_guard` in listing 2.3 first tests to see if the `std::thread` object is `joinable()` ❶ before calling `join()` ❷. This is important, because `join()` can be called only once for a given thread of execution, so it would therefore be a mistake to do so if the thread had already been joined.

The copy constructor and copy-assignment operator are marked `=delete` ❸ to ensure that they're not automatically provided by the compiler. Copying or assigning such an object would be dangerous, because it might then outlive the scope of the thread it was joining. By declaring them as deleted, any attempt to copy a `thread_guard` object will generate a compilation error. See appendix A, section A.2, for more about deleted functions.

If you don't need to wait for a thread to finish, you can avoid this exception-safety issue by *detaching* it. This breaks the association of the thread with the `std::thread` object and ensures that `std::terminate()` won't be called when the `std::thread` object is destroyed, even though the thread is still running in the background.

2.1.4. Running threads in the background

Calling `detach()` on a `std::thread` object leaves the thread to run in the background, with no direct means of communicating with it. It's no longer possible to wait for that thread to complete; if a thread becomes detached, it isn't possible to obtain a `std::thread` object that references it, so it can no longer be joined. Detached threads truly run in the background; ownership and control are passed over to the C++ Runtime Library, which ensures that the resources associated with the thread are correctly reclaimed when the thread exits.

Detached threads are often called *daemon threads* after the UNIX concept of a *daemon process* that runs in the background without any explicit user interface. Such threads are typically long-running; they may well run for almost the entire lifetime of the application, performing a background task such as monitoring the filesystem, clearing unused entries out of object